Questions on Stack and Queue Adapter

Stack Adapter Practice Questions

Question 1: Basic Stack Operations

Write a Java program that demonstrates the basic operations of a Stack:

```java
import java.util.Stack;

public class StackExample {
    public static void main(String[] args) {
        Stack<Integer> stack = new Stack<>();

        // Push elements
        stack.push(10);
        stack.push(20);
        stack.push(30);

        System.out.println("Stack: " + stack);
        System.out.println("Top element: " + stack.peek());
        System.out.println("Popped element: " + stack.pop());
        System.out.println("Stack after pop: " + stack);
        System.out.println("Is stack empty? " + stack.isEmpty());
    }
}
```

Question 2: Stack for Parentheses Matching

How would you use a Stack to check if parentheses are balanced in a string?

```java
import java.util.Stack;

public class ParenthesesChecker {
    public static boolean isBalanced(String expression) {
        Stack<Character> stack = new Stack<>();

        for (char ch : expression.toCharArray()) {
            if (ch == '(' || ch == '[' || ch == '{') {
                stack.push(ch);
            } else if (ch == ')' || ch == ']' || ch == '}') {
                if (stack.isEmpty()) return false;

                char top = stack.pop();
                if (!isMatchingPair(top, ch)) {
                    return false;
                }
            }
        }
        return stack.isEmpty();
    }

    private static boolean isMatchingPair(char open, char close) {
        return (open == '(' && close == ')') ||
               (open == '[' && close == ']') ||
               (open == '{' && close == '}');
    }
}
```

Question 3: Stack for Reversing Elements

Demonstrate how to use a Stack to reverse a string:

```java
import java.util.Stack;

public class StringReverser {
    public static String reverseString(String input) {
        Stack<Character> stack = new Stack<>();

        // Push all characters to stack
        for (char ch : input.toCharArray()) {
            stack.push(ch);
        }

        // Pop all characters to build reversed string
        StringBuilder reversed = new StringBuilder();
        while (!stack.isEmpty()) {
            reversed.append(stack.pop());
        }

        return reversed.toString();
    }
}
```

Queue Adapter Practice Questions

Question 4: Basic Queue Operations

Write a Java program demonstrating Queue operations using LinkedList:

```java
import java.util.Queue;
import java.util.LinkedList;

public class QueueExample {
    public static void main(String[] args) {
        Queue<String> queue = new LinkedList<>();

        // Add elements (enqueue)
        queue.offer("First");
        queue.offer("Second");
        queue.offer("Third");

        System.out.println("Queue: " + queue);
        System.out.println("Front element: " + queue.peek());
        System.out.println("Removed element: " + queue.poll());
        System.out.println("Queue after removal: " + queue);
        System.out.println("Queue size: " + queue.size());
    }
}
```

Question 5: Priority Queue Implementation

How do you implement a Priority Queue for processing tasks based on priority?

```java
import java.util.PriorityQueue;
import java.util.Comparator;

class Task {
    String name;
    int priority;

    Task(String name, int priority) {
        this.name = name;
        this.priority = priority;
    }

    @Override
    public String toString() {
        return name + " (Priority: " + priority + ")";
    }
}

public class TaskScheduler {
    public static void main(String[] args) {
        PriorityQueue<Task> taskQueue = new PriorityQueue<>(
            Comparator.comparingInt(task -> task.priority)
        );

        taskQueue.offer(new Task("Low Priority Task", 3));
        taskQueue.offer(new Task("High Priority Task", 1));
        taskQueue.offer(new Task("Medium Priority Task", 2));

        while (!taskQueue.isEmpty()) {
            System.out.println("Processing: " + taskQueue.poll());
        }
    }
}
```

Question 6: Queue for BFS Implementation

Demonstrate how Queue is used in Breadth-First Search:

```java
import java.util.*;

public class BFSExample {
    private Map<Integer, List<Integer>> adjacencyList;

    public BFSExample() {
        adjacencyList = new HashMap<>();
    }

    public void addEdge(int source, int destination) {
        adjacencyList.computeIfAbsent(source, k -> new ArrayList<>()).add(destination);
        adjacencyList.computeIfAbsent(destination, k -> new ArrayList<>()).add(source);
    }

    public void bfs(int startVertex) {
        Set<Integer> visited = new HashSet<>();
        Queue<Integer> queue = new LinkedList<>();

        visited.add(startVertex);
        queue.offer(startVertex);

        while (!queue.isEmpty()) {
            int vertex = queue.poll();
            System.out.print(vertex + " ");

            for (int neighbor : adjacencyList.getOrDefault(vertex, new ArrayList<>())) {
                if (!visited.contains(neighbor)) {
                    visited.add(neighbor);
                    queue.offer(neighbor);
                }
            }
        }
    }
}
```

Stack vs Queue Comparison Questions

Question 7: When to Use Stack vs Queue

Compare the use cases for Stack and Queue:

Stack (LIFO - Last In, First Out):

- Undo operations in applications
- Function call management
- Expression evaluation
- Backtracking algorithms

Queue (FIFO - First In, First Out):

- Task scheduling
- Breadth-first search
- Print job management
- Buffer for data streams

Question 8: Converting Between Stack and Queue

How can you implement a Queue using two Stacks?

```java
import java.util.Stack;

public class QueueUsingStacks<T> {
    private Stack<T> stack1;
    private Stack<T> stack2;

    public QueueUsingStacks() {
        stack1 = new Stack<>();
        stack2 = new Stack<>();
    }

    public void enqueue(T item) {
        stack1.push(item);
    }

    public T dequeue() {
        if (stack2.isEmpty()) {
            while (!stack1.isEmpty()) {
                stack2.push(stack1.pop());
            }
        }
        return stack2.isEmpty() ? null : stack2.pop();
    }

    public boolean isEmpty() {
        return stack1.isEmpty() && stack2.isEmpty();
    }
}
```

Multiple Choice Quiz Questions

1. Which method is used to add an element to the top of a Stack? a) add() b) push() c) offer() d) insert()

Answer: b) push()

2. What does the peek() method do in a Stack? a) Removes and returns the top element b) Returns the top element without removing it c) Checks if the stack is empty d) Returns the size of the stack

Answer: b) Returns the top element without removing it

3. Which interface does Stack class implement in Java? a) List b) Queue c) Both List and Queue d) Collection

Answer: a) List

4. In a Queue, which method is preferred for adding elements? a) add() b) push() c) offer() d) insert()

Answer: c) offer()

5. What is the time complexity of push() and pop() operations in a Stack? a) O(n) b) O(log n) c) O(1) d) $O(n^2)$

Answer: c) O(1)

Practice Coding Questions

**1. Implement a method that uses a Stack to evaluate a postfix expression. The method should take a string containing numbers and operators (+, -, , /) separated by spaces and return the result.*

2. Create a class that implements a circular queue using an array. Include methods for enqueue, dequeue, isEmpty, isFull, and display. The queue should handle the wraparound condition properly.

3. Write a program that uses a Stack to convert an infix expression to postfix notation. Handle parentheses and operator precedence correctly.

4. Implement a method that checks if a given string of brackets (including [], {}, and ()) is properly nested using a Stack. Return true if properly nested, false otherwise.

5. Create a class that simulates a print queue where print jobs have different priorities. Use a PriorityQueue to ensure high-priority jobs are processed first. Include methods to add jobs, process the next job, and display the current queue status.